

Kafi Admin Dashboard — Production-Readiness Audit

Nannies/Ops · Families/Business · Platform · Backend — analysis only

Kafi Admin Dashboard — Production-Readiness Audit

Platform: React + TypeScript admin panel (`admin-panel/`) — the admin role, and the admin↔user (nanny/family) actions, plus the backend (Cloud Functions / Firestore rules / indexes) where it breaks the dashboard.

Method: multi-auditor code trace, cross-checked against the app's consumption and the Firestore rules. Every finding is anchored to `file:line`, rated by severity, and stated as a concrete **failing use case**, including whether an admin action actually **propagates** to the affected user. **Date:** 2026-07-22 · **Mode:** analysis only (no code changed).

How to read this

[SEVERITY] title — file:line — Failing use case — Fix. **CRITICAL** = blocks launch or a security hole; **MAJOR** = a real admin capability is broken or fails silently; **MINOR** = correctness/UX gap. **CONFIRMED** = verified in code (some by running `tsc`); **SUSPECTED** = depends on runtime/deploy data.

Verified baseline (important): an auditor initially reported the panel "cannot build (**CRITICAL**)". I ran the toolchain to check: `npm run build (tsc && vite build)` **exits 0 and the panel deploys fine today**. The real issue is subtler and confirmed below — the `tsc` step type-checks **nothing** because the root `tsconfig.json` is a solution/references file, so type errors ship silently (`npm tsc -b` surfaces 5 real ones). That false "build broken" is therefore **recharacterized** into the vacuous-type-check **Major** (§2) + a stale-mock **Minor**.

Propagation baseline — CONFIRMED WORKING (so the findings below are precise): approve/reject nanny, per-document approve/reject, intro-video approve/reject, block/unblock (nanny & family), dispute reply/resolve, ticket reply, and the new hide-inactive-nannies toggle **all reach the affected user** end-to-end via `onDocumentReviewed/onNewDisputeMessage/onDisputeResolved/onNewTicketMessage` + the app's readers/streams. These are sound — the findings are the gaps around them.

Executive summary

The dashboard's core moderation actions work and propagate correctly. The production risks cluster in three themes:

- **1 Critical (security)** — a public, unauthenticated bootstrap endpoint creates the first admin with a **hardcoded default password**, so anyone can seize `superAdmin`.
- **~14 Major** — the CI type-check is vacuous (type bugs ship silently); platform-wide counts + **revenue/VAT are capped at 200 rows and computed from mismatched sources** (financially misleading); Broadcast blasts every device with **no confirmation**; several safety/ops screens **mask load failures as "empty"** (an admin believes there are zero disputes/tickets when the read failed); resolving a **ticket doesn't notify the user**; and admins **cannot actually change a subscription**.
- **~14 Minor** — error-state, mock-vs-live, labeling, and rules gaps.

Root causes (see §4): (1) **inconsistent error/loading handling** — the correct pattern exists in-repo but wasn't ported to the safety/ops queues and detail pages; (2) **mock-vs-live divergence** on the ops board (mock auto-advances ticket/dispute status, live doesn't); (3) **platform aggregates derived from capped fetches** rather

than server-side counts; (4) a **genuine propagation gap** — ticket status changes have no notifying Cloud Function.

1. Nannies · Operations · Safety

Safety — Disputes (highest-impact cluster)

[MAJOR] `DisputeDetail` hangs on "Loading..." forever on any read failure — **CONFIRMED** — `pages/disputes/DisputeDetail.tsx:58-65` Failing use case: an admin opens a dispute whose read fails (permission/network/missing index) and the page spins indefinitely with no error, because `load()` awaits `DisputeService.get/listMessages` with **no try/catch/finally** — the identical bug was already fixed in `TrialDetail.tsx:141-145` (with a comment) but never ported here. Fix: try/catch/finally + error state.

[MAJOR] Dispute reply / resolve failures are silent and lock the UI — **CONFIRMED** — `pages/disputes/DisputeDetail.tsx:90-97, 99-106` Failing use case: an admin replies to (or resolves) a dispute and the write rejects → `sending/busy` stays true forever (composer/button disabled), no error shows, the typed reply/resolution is lost with no retry, because `send()/resolve()` await with no catch. (On success it does propagate to the reporter.) Fix: try/catch/finally, surface the error, preserve the text.

[MAJOR] The Disputes queue shows "No disputes filed" when the load actually FAILED — **CONFIRMED** — `pages/disputes/Disputes.tsx:33-37` Failing use case: the query fails, `.then(setItems).finally()` has no `.catch`, so `items` stays `[]` and the safety queue renders the empty state — an admin believes there are **zero** abuse/fraud/no-show reports to action when reports exist and go unhandled. Fix: `.catch` → a distinct error state (the in-repo reference is `AllNannies.tsx:48-53`).

[MAJOR] "Investigating" is unreachable in production; a live reply never advances status — **CONFIRMED** — `services/firestore.ts:1330-1336 (live) vs 1323-1327 (mock); UI DisputeDetail.tsx:182-213` Failing use case: an admin starts working a dispute (replies) in prod, but status stays `open` — live `sendMessage` only adds the message, `DisputeDetail` offers only terminal Resolved/Dismissed, and rules make dispute status admin-only. Disputes jump `open` → `resolved`, the "Investigating" stat is permanently 0, and untouched vs in-progress reports are indistinguishable (mock auto-advances, hiding it in dev). Fix: advance to `investigating` on the first admin reply in live `sendMessage`, or add a "Mark investigating" control.

[MINOR] Resolution notes labeled "visible to both parties" but only the reporter is notified/can read — **CONFIRMED** — `DisputeDetail.tsx:204, functions/src/triggers/dispute.ts:50, 62, firestore.rules:262-272` Failing use case: an admin resolves a dispute with an outcome affecting the reported user; that user gets no push/inbox (`onDisputeResolved` targets `reporterId` only) and can't read the support chat — the "both parties" promise is false for the reported side. Fix: notify `reportedUserId` on resolution, or reword the placeholder.

Safety — Support tickets

[MAJOR] The SupportTickets queue swallows load errors and shows "No tickets yet." — **CONFIRMED** — `pages/support/SupportTickets.tsx:34-39` Failing use case: the query fails and `.catch(() => setItems([]))` silently swallows it, rendering the same empty state as a genuinely empty queue — an admin believes there are no open requests when loads failed, and real tickets (double-charges, trial help) go unanswered. Fix: set + render an error state.

[MAJOR] Resolving/closing a ticket does NOT notify the opener — no Cloud Function on ticket status change — **CONFIRMED** — `services/firestore.ts:1351-1358; functions/src/index.ts (only onNewTicketMessage)` Failing use case: an admin marks a ticket `resolved/closed` without also sending a message → the opener is never told (unlike disputes, which have `onDisputeResolved`, tickets have **no**

`onDocumentUpdated` trigger). The status reaches Firestore but is silent unless the user reopens the app and re-reads the thread. Fix: add an `onTicketStatusChanged` CF mirroring `onDisputeResolved`, or have `updateStatus` post a system message.

[MINOR] Ticket reply doesn't auto-advance `open→investigating` in live (mock does) — CONFIRMED — `services/firestore.ts:1398-1409` vs mock `1392-1395` Failing use case: replying to an open ticket in prod leaves it `open` and the "In progress" count static unless the admin also clicks a status button (less severe than the dispute case — `SupportTicketDetail` has an explicit status control). Fix: advance on first reply, or document the intent.

Trials

[MINOR] AllTrials masks a load failure as an empty result — CONFIRMED — `pages/trials/AllTrials.tsx:28-32` Failing use case: `TrialService.listAll` rejects (missing index/permission), `.then(setItems).finally()` has no `.catch`, so the oversight page shows "No trials match your filters." and the admin believes no trials are running (read-only, lower stakes; `TrialDetail` is correctly guarded). Fix: `.catch` + error state.

Nannies

[MINOR] Document-status filter offers values that never exist in production (mock-vs-live enum mismatch) — CONFIRMED — `pages/nannies/VerifyDocuments.tsx:21, 31-36` Failing use case: filtering the queue by "resubmitted"/"pending" is always empty because the app's real `DocumentStatus` enum is `{missing, uploaded, reviewing, approved, rejected}` (`nanny_model.dart:96`) — those are mock-only labels, and real `reviewing` docs have no filter option. Fix: align `DOC_STATUSES/docVariant` with the app enum.

[MINOR] AllNannies Block/Unblock failure is silent — CONFIRMED — `pages/nannies/AllNannies.tsx:56-68` Failing use case: an admin clicks Block on a list row, the write fails, and the `try/finally` has no `catch` → no signal; the admin believes a nanny is blocked when she isn't (`NannyDetail.toggleBlock` correctly alerts — this path is inconsistent). Fix: `catch` → toast.

[MINOR] Dead, divergent `listPendingVideos`; `VerifyDocuments` labels/count say "documents" but count nannies — CONFIRMED — `services/firestore.ts:727-734`, `pages/nannies/VerifyDocuments.tsx:201, 210, 79-82` Failing use case: `listPendingVideos` has zero callers and its mock/live predicates diverge; and the `VerifyDocuments` header reads "{n} documents pending" where n is nannies (status `pending`), with a comment implying a non-existent auto-approve. Cosmetic/omission (approve/reject still propagate). Fix: correct labels/comment; remove or wire the dead method.

2. Families · Business · Platform / Infra

Platform / Infra

[CRITICAL · security] Public, unauthenticated admin-bootstrap creates the first admin with a hardcoded default password — CONFIRMED — `functions/src/utils/ensureFirstAdmin.ts:19` ('Kafi@Admin2026!'), `functions/src/bootstrapAdmin.ts:5-6` (`invoker: 'public'`), invoked from `pages/Login.tsx:27` Failing use case: when `admins` is empty and no `KAFI_ADMIN_BOOTSTRAP_PASSWORD` is set, the first caller of the public `bootstrapFirstAdmin` endpoint creates `admin@kafi.ae` with the in-source password and can immediately sign in — a first-caller race and a secret-in-code, and anyone who reads the repo can log in afterward with the known default. Fix: require the env password (fail if unset), disable/delete the endpoint after first use, rotate the password.

[MAJOR] The CI "type-check" is vacuous — type errors ship silently — CONFIRMED (ran the toolchain)

— `package.json:8` ("`build`": "`tsc && vite build`"), `tsconfig.json` (solution file: `files:[]` + references) Failing use case: `npm run build` runs bare `tsc`, which on a solution/references `tsconfig.json` type-checks **nothing** (exit 0); only `tsc -b` builds the referenced projects. `npm run tsc -b` reports **5 real TS2353 errors** (`services/firestore.ts:372,403,432,464,496` — mock nanny `stats` still set `averageRating/reviewsCount`, removed from `NannyStatsRow` when the reviews pipeline was retired). CI + local builds are green anyway, so **any type error in src/ reaches production unnoticed**. Fix: change the build to `tsc -b && vite build` (and fix the 5 surfaced errors — the stale-mock **Minor** below).

[MAJOR] Platform-wide counts + revenue/VAT are capped at 200 rows — CONFIRMED — `services/firestore.ts:719` (nannies limit 200), `:867` (families limit 200) Failing use case: past 200 families, `AllFamilies`, the Dashboard "Total users"/"Active subs"/"New today" (`Dashboard.tsx:200-207`), the Subscriptions counts, the Sidebar badges, and — critically — `RevenueService.summary()` MRR/VAT (which iterates the capped `FamilyService.list()`, `firestore.ts:1584-1611`) **all under-report with no indication**. Revenue is undercounted the moment there are >200 active subscribers. Fix: paginate or aggregate server-side (`getCountFromServer` / maintained counters); never derive platform totals from `limit(200)`.

[MAJOR] No 404 / catch-all route — CONFIRMED — `App.tsx:43-60` Failing use case: any unknown path under `/` (typo/stale bookmark/deleted-entity link) matches nothing, so `Layout` renders with a **blank content area** — sidebar visible, main empty, no redirect. Fix: add `<Route path="*" element={<Navigate to="/" replace /> />` (or a `NotFound` page).

[MINOR] Misconfigured prod deploy fails silently instead of fast — SUSPECTED — `config/firebase.ts:8-13` Failing use case: with `VITE_USE MOCK=false` but missing `VITE_FIREBASE_*`, `initializeApp` succeeds with placeholder `'mock-api-key'/'projectId:'kafi-mock'`, `db` is non-null, and every query hits a nonexistent project; the app "looks up" but nothing loads (Sidebar badge `catch` is empty). Fix: in non-mock mode, throw at startup if any required env var is absent.

Auth gate — no authorization hole (verified): `ProtectedRoute` is client-only, but `toAdminUser` re-verifies `admins/{uid}` and signs out non-admins, and Firestore rules enforce `isAdmin()` server-side for every sensitive collection — a forged `localStorage` renders the shell but every read is denied (empty pages, no leak). No admin query needs a missing composite index.

Families

[MAJOR] Block/unblock in the families list fails silently — CONFIRMED — `pages/families/AllFamilies.tsx:45-57` Failing use case: an admin clicks Block/Unblock in the list and the write fails; the `try/finally` has no `catch`, so the optimistic row update never runs and the rejection is swallowed — no toast, no revert, no error, and the admin may assume the block took (`FamilyDetail.tsx:164-175` does this correctly). Fix: add a `catch` that surfaces + reverts. (*Block itself propagates correctly to the app when it succeeds.*)

[MAJOR] Admin cannot actually change a subscription or reset free contacts — CONFIRMED — `services/firestore.ts:876` (`overrideSubscription`), `:892` (`resetFreeContacts`); zero UI callers Failing use case: to comp a subscription, fix a mis-charged plan, or reset a family's free-contact counter, there is **no control anywhere** — `FamilyDetail`'s subscription block is explicitly "view only" (`FamilyDetail.tsx:264-275`). The data layer supports these writes but they're dead code, so a core "manage subscriptions" capability is missing. Fix: wire override/reset into `FamilyDetail` (or remove if out of scope).

[MINOR] Free-contacts remaining hardcoded `'/ 5'` — CONFIRMED — `pages/families/FamilyDetail.tsx:272` Failing use case: if the operator changes `settings.freeContactLimit` away from 5, this field still renders `"X / 5"`, misreporting the family's remaining free contacts. Fix: read the limit from `SettingsService.get()`.

[MINOR] Conversation message load has no error state — CONFIRMED — `components/chat/ConversationsPanel.tsx:53-55` Failing use case: opening a family↔nanny thread whose `listMessages` throws leaves an empty thread with no error — indistinguishable from "no messages". Fix: `catch` → error state in the modal.

Business — Revenue / Subscriptions / Broadcast

[MAJOR] Dashboard and Revenue report different revenue from different sources — CONFIRMED —
Dashboard `Dashboard.tsx:179,185-187,282-287` (subscription-derived MRR via `RevenueService.summary()`, `firestore.ts:1598-1611`) vs **Revenue** `Revenue.tsx:113,132-135` (`transactions`)
Failing use case: with billing not live (empty `transactions`), the Revenue page honestly shows "Billing is not integrated yet" while the Dashboard simultaneously shows "This month revenue AED " from active-subscription plan prices — two pages contradict each other, and neither number is actual collected cash. Fix: one source of truth (derive Dashboard from `transactions`, or relabel "Projected MRR").

[MAJOR] "VAT collected → FTA" is 5% of *projected* MRR, not collected VAT — CONFIRMED —
Dashboard `Dashboard.tsx:284-285,307`; **value from** `firestore.ts:1611` (`Math.round(monthly * 0.05)`) Failing use case: an admin reads "VAT collected" to gauge FTA liability, but it's 5% of not-yet-collected subscription value (including families who never paid a transaction) — financially misleading. Fix: compute VAT from paid `transactions` (as the Revenue page does) and relabel.

[MAJOR] Broadcast has no confirmation before blasting every device — CONFIRMED — `pages/business/Broadcast.tsx:99-101`; `BroadcastService.send` → `firestore.ts:1417-1431` → `onBroadcastCreated` **FCM fan-out** Failing use case: with "All users" selected, one click (or a premature Enter) pushes a notification to the **entire user base** — no "are you sure?", no reach preview, no undo. Fix: a confirm dialog showing audience + estimated reach before writing the broadcast doc.

[MINOR] "Subscribers" audience excludes cancelled-but-in-period families — CONFIRMED —
`functions/src/triggers/broadcast.ts:28-33` (`status=='active'` only) vs **admin UI counting active+cancelled as subscribed** (`AllFamilies.tsx:67`) Failing use case: a "Subscribers only" broadcast skips families with status `cancelled` (still paid-through) even though every admin surface labels them "Subscribed". Fix: align the CF audience (or relabel "Active subscribers only").

[MINOR] Recent-broadcasts panel always empty in demo — CONFIRMED — `firestore.ts:1433` (`listRecent` returns `[]` in mock) — the `send`→`history` loop looks broken in demos. Fix: record sent mock broadcasts in-memory.

Dashboard

[MINOR] Mock `byPlan`/MRR is fabricated and shown as real — CONFIRMED (mock-only) —
`firestore.ts:1573-1579` rendered at `Dashboard.tsx:290-313` Failing use case: in mock/demo the Dashboard shows ~65 subscribers + AED 10,800 MRR while only 4 mock families (2 active) exist — internally inconsistent in any screenshot/demo. Fix: derive the mock summary from `mockFamilies/mockTransactions`.

[MINOR] Duplicate family fetches per page load — CONFIRMED — `Dashboard.tsx:175-181` + `Subscriptions.tsx:25` each call `FamilyService.list()` and `RevenueService.summary()` (which calls `FamilyService.list()` again, `firestore.ts:1585`) — doubles reads/latency on every load. Fix: pass the fetched array into `summary()` or memoize.

Settings (new page) — clean

The hide-inactive toggle is well-built: load with cancellation guard + error state, optimistic set with revert-on-failure, disabled-while-busy switch (`Settings.tsx:16-72`); `SettingsService.get/update` merge defaults to `settings/global`; rules restrict write to admin (`firestore.rules:308`); the app reads it and filters on `lastActiveAt` with a safe "unknown ≠ inactive" rule. End-to-end sound — no defects found.

3. Backend / platform (breaks or blocks the

dashboard)

[MINOR · but a latent blocker] `transactions` collection has no Firestore rule — **CONFIRMED** — read at `services/firestore.ts:1533,1554,1586`; **absent from `firestore.rules`** Failing use case: if `transactions` is ever populated (real billing), the admin revenue dashboard's reads are **denied** by default — Revenue/Dashboard break the moment billing goes live. Passes today only because nothing writes the collection. Fix: add an `isAdmin()`-gated rule.

[MINOR] RevenueCat webhook has no event dedup / out-of-order guard — **CONFIRMED** — `functions/src/triggers/webhook.ts:53-144` Failing use case: a duplicate `RENEWAL` resets `subscription.startDate` to now; a late `EXPIRATION` after a `RENEWAL` wrongly expires an active sub — corrupting the subscription numbers every admin revenue/subscription surface reports. Fix: record processed `event.id`s; ignore events older than stored state.

[MAJOR] Account-deletion cascade omits `disputes/tickets` — the admin safety queues accumulate orphans + residual PII — **CONFIRMED** — `functions/src/triggers/delete.ts` Failing use case: when a user is deleted, their `tickets/disputes` (+ message subcollections, `reporterId`/description PII) persist, so the admin Disputes/Support queues show rows for non-existent users and PII survives a "deleted" account. Fix: extend the cascade (by `reporterId/reportedUserId/openerId`) + subcollections; decide retain-for-audit + anonymize where legally required.

(The missing `families.subscription` composite index and the `syncMockSubscription` bypass — both in the mobile report — also degrade admin subscription/revenue accuracy; cross-referenced there.)

4. Recurring root causes

1. **Inconsistent error/loading handling.** The correct pattern exists in-repo (`AllNannies.tsx:48-53` `loadError` state; `TrialDetail.tsx:141-145` `try/catch/finally`) but was **not ported** to `Disputes`, `SupportTickets`, `AllTrials` (mask failures as empty) or `DisputeDetail` (hangs on load; locks on reply/resolve). → most of §1's Majors.
2. **Mock-vs-live status divergence on the ops board.** Mock auto-advances `open→investigating`; live doesn't — for disputes this makes 'investigating' unreachable; for tickets the queue count only moves on explicit status change.
3. **Platform aggregates derived from capped fetches.** Counts + revenue + VAT come from `limit(200)` client reads and mismatched sources, not server-side counts. → the financial-accuracy Majors.
4. **One true propagation gap + one security hole.** Ticket status changes have no notifying CF (disputes do); and the public bootstrap ships a default admin password.

5. Consolidated priority order (web dashboard)

Rank	Finding	Where
1	Public bootstrap + hardcoded default admin password (seize <code>superAdmin</code>)	\$2 · <code>ensureFirstAdmin.ts:19</code> , <code>bootstrapAdmin.ts:5</code>
2	Vacuous CI type-check → type errors ship silently (<code>tsc</code> should be <code>tsc -b</code>)	\$2 · <code>package.json:8</code> <code>tsconfig.json</code>
3	Revenue/VAT wrong: capped at 200 rows + mismatched sources + projected-not-collected	\$2 · <code>firestore.ts:867,1598-1611</code> , <code>Dashboard.tsx:282</code>

4	Safety queues mask load failures as "empty" (disputes/tickets go unhandled)	§1 · Disputes.tsx:33 , SupportTickets.tsx:34
5	DisputeDetail hangs on load + locks on reply/resolve (silent)	§1 · DisputeDetail.tsx:58-106
6	Broadcast blasts all users with no confirmation	§2 · Broadcast.tsx:99
7	Ticket resolve/close doesn't notify the user (no CF) + deletion leaves ticket/dispute orphans	§1/§3 · firestore.ts:1351 , delete.ts
8	Admin can't change a subscription / reset free contacts (dead methods)	§2 · firestore.ts:876,892
9	"Investigating" unreachable in prod (live status never advances) + block-list silent failures	§1/§2
10	transactions has no rule (Revenue breaks when billing is live) + webhook dedup	§3
—	Then the Minor sweep (mock-vs-live labels, 404 route, hardcoded /5, i18n)	§1-3

Every finding above is anchored to [file:line](#); the "build broken" report was verified false and recharacterized. No code was changed producing this audit.